Triton

A Contracts-First, Deterministic Mermaid-Superset Diagram Compiler

Architecture & Specification

Triton Project

June 2026

Scope of this document. This specification describes Triton *as it is implemented*. Every construct, contract, and example here corresponds to shipped code under `src/` and runnable sources under `examples/` (each `.mmd` file beside its rendered `.svg`). It is deliberately short: where the implementation is the source of truth, the document points at it rather than restating it. Features that are not built are listed explicitly in §0.8, not described as if they were. **The figures are dogfooded:** every diagram below is rendered by Triton itself from an example source (regenerate with `pnpm figures`).

Contents

0.1 Overview

Triton is a deterministic diagram compiler. It takes a small textual source and produces a single, byte-stable SVG (optionally rasterised to PNG). Its surface syntax is a *superset of Mermaid*: existing Mermaid sources render unchanged, and Triton adds diagram families Mermaid does not have.

0.1.1 The central claim

Every diagram kind — whether a Mermaid flowchart or a Triton-native AVL tree — is a `DiagramModule`: a unit that parses its own source into its own intermediate representation (IR), lays that IR out, and emits a shared `Scene`. A single renderer turns any `Scene` into SVG. There is no "god IR": each diagram owns the data model that fits it, and they meet only at the `Scene` contract.

This contracts-first design is what makes the system both broad and consistent. Adding a diagram kind means writing a parser and a layout function against fixed contracts; it never means touching the renderer, the theme system, or other diagrams.

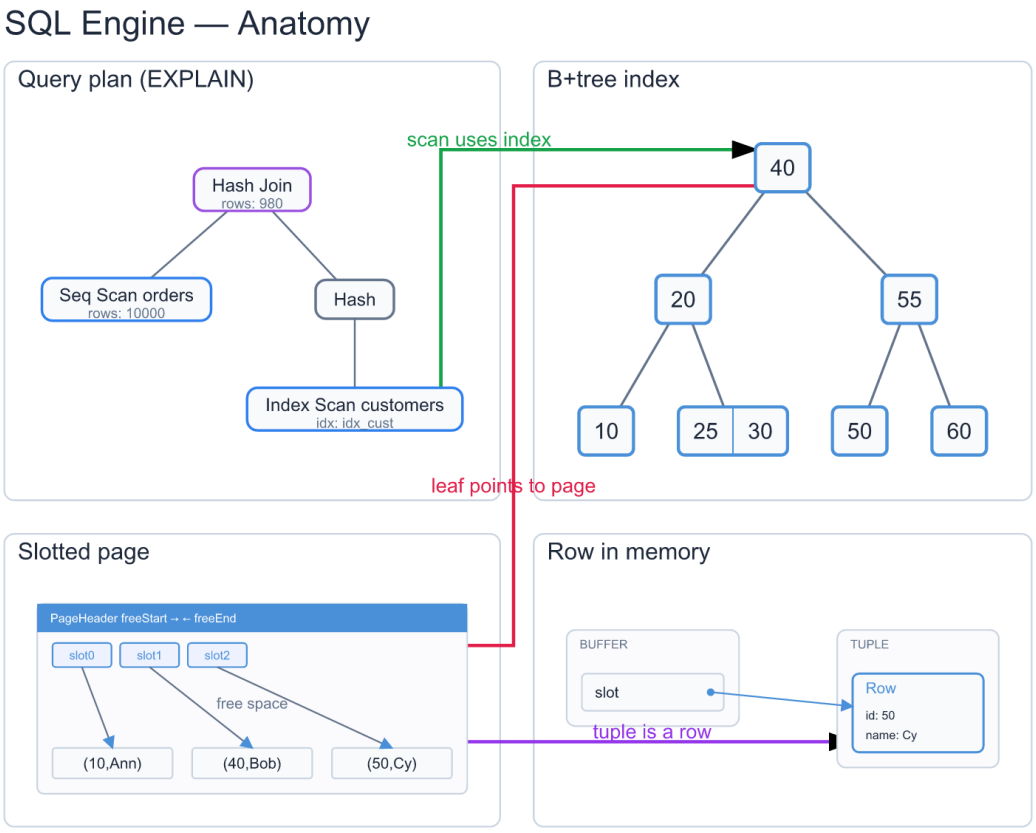

Figure 1: An "SQL engine anatomy" poster, authored as a single Triton source: a query plan, a B+tree index, a slotted storage page, and an in-memory row, composed in one grid and joined by cross-diagram links. This combination — composing heterogeneous diagrams and linking nodes across them — is what Triton adds beyond Mermaid.

0.1.2 What Triton renders

- **Mermaid-compatible kinds** — flowchart, sequence, state, class, ER, gantt, journey, mindmap, gitgraph, pie, xychart, quadrant, radar, sankey, kanban, requirement, block, c4, architecture, packet, timeline.
- **Tree family** (Triton-native) — a decorated `tree` engine plus value-driven frontends `plan`, `avl`, `rbtree`, `btree`, `radix`, `segtree`, `heap`. These build *correct-by-construction* structures.
- **Struct / memory family** — `array`, `linkedlist`, `memory` (regions with cross-region pointers), `page` (a slotted storage page).
- **Topology** — cost-tiered node graphs with legends and nested groups.
- **Poster composition** — a grid that embeds any of the above as cells, with cell spanning and *cross-diagram links* between panels.

0.1.3 Design properties

Contracts-first.
The `Scene` is the one rendering contract; the `DiagramModule` is the one diagram contract.

Per-diagram IR.
Each kind models its own domain; the god-IR is explicitly rejected.

Grammar = semantics, theme = style.
Source decides *what* a diagram means; the theme decides *how* it looks. They never mix.

Deterministic.
The same source plus theme always yields the same SVG, byte for byte (§0.7).

Correct-by-construction.
The value-driven builders (e.g. `avl`) compute valid structures from operations; you cannot author an invalid one.

0.1.4 What Triton is not

Triton is a text-to-SVG compiler, nothing more. It does *not* ingest data sources, does *not* construct diagrams from natural-language prompts, does *not* export PPTX, and has no general animation pipeline. The full out-of-scope list is in §0.8.

0.2 Architecture

The compiler is a short pipeline with three contracts at its joints: *detection*, the `DiagramModule`, and the `Scene`.

0.2.1 The pipeline

1. **Detect** (`src/frontend/detect.ts`). The source is classified by its leading header into an input format (`mermaid` text or `yaml`/JSON) and a `DiagramKind` (e.g. a line

starting `flowchart` → `flowchart`; a line starting `avl` → `avl`).

2. **Dispatch** (`src/frontend/registry.ts`). The kind selects a registered `DiagramModule`.
3. **Parse**. The module turns source into its own IR (`parseMermaid` for text, `parseYaml` for the structured alternate).
4. **Layout**. The module places its IR and returns a `LayoutResult` — a `Scene` plus a node-anchor registry.
5. **Render** (`src/render/svg.ts`). One renderer turns the `Scene` into SVG; `resvg` optionally rasterises it to PNG.

Text is the primary surface. The `yaml`/JSON path is a thin alternate input (each module's `parseYaml` accepts a pre-built IR); every example in this repository is authored as Mermaid-superset text (`.mmd`).

0.2.2 The `DiagramModule` contract

Every diagram kind implements one interface (`src/contracts/diagram.ts`):

```
interface DiagramModule<IR extends BaseIR> {
  parseMermaid(input: string): IR;
  parseYaml(input: string): IR;
  layout(ir: IR, theme: ResolvedTheme): Promise<LayoutResult>;
}
```

`BaseIR` carries only what the pipeline needs uniformly (version, metadata, optional overlays and per-diagram theme override). Each kind extends it with its own fields. `layout` returns:

```
interface LayoutResult {
  scene: Scene;                  // what the renderer draws
  anchors: NodeAnchorRegistry;  // node bounds + cardinal ports
}
```

The anchor registry is what makes a diagram *linkable*: it exposes each node's bounding box (and optional N/S/E/W ports) in the diagram's local coordinate space, so a composition layer can draw arrows between nodes that live in different diagrams (§0.6).

0.2.3 Registries

Three small in-process registries hold the moving parts: diagram modules (`registerDiagram`), renderers (`registerRenderer`), and edge routers. Registration is the only wiring needed to add a kind; the rest of the system discovers it by contract.

0.2.4 Build

The project is a single package at the repository root. Grammars are written in Peggy (one `grammar.peggy` per diagram under `src/diagrams/<kind>/`); `scripts/build-grammars.mjs` compiles them to parsers, and `tsc` compiles the TypeScript. Tests run under Vitest.

0.3 The Scene Contract and Renderer

The `Scene` is the single rendering contract. Diagram layout engines speak only `Scene`; the renderer understands only `Scene`. Neither knows about the other's specifics.

0.3.1 Scene and its elements

A `Scene` (`src/contracts/scene.ts`) is a viewBox, an optional background, a flat list of elements, and optional raw `defs` (e.g. arrow markers):

```
interface Scene {
  viewBox: { x: number; y: number; width: number; height: number };
  background?: Color;
  elements: SceneElement[];
  defs?: string[];
}
```

`SceneElement` is a five-variant union — the entire vocabulary the renderer must support:

Variant	Carries
`rect`	bounds, fill, stroke, stroke width, optional corner radius
`circle`	centre, radius, fill, stroke
`path`	SVG path `d`, stroke, fill, dash, markers, animation
`text`	content, position, font size/family, fill, anchor, weight
`group`	children, optional transform / id / opacity

That deliberately small surface is why one renderer covers every diagram: a tree, a slotted page, and a flowchart all reduce to rectangles, circles, paths, and text.

0.3.2 The Pen

Layout code does not build element objects by hand. A `Pen` (`src/scene/build.ts`), bound once to a theme, produces elements with the theme's font family defaulted and the `type` discriminant removed:

```
const p = pen(theme);
p.rect(bounds, fill, stroke, strokeWidth, { rx });
p.circle(center, radius, fill, stroke, strokeWidth);
p.path(d, stroke, strokeWidth, { dash, markerEnd, animated });
p.text(content, x, y, size, fill, { anchor, weight });
```

0.3.3 The renderer

`renderSVG(scene)` (`src/render/svg.ts`) emits the SVG string directly — one element at a time, in document order, with XML-escaped text and fixed numeric formatting. There is exactly one backend. PNG, when needed, is the same SVG rasterised by `resvg`; there is no Skia, PPTX, PDF, or HTML backend.

0.4 Shared Kernels

Diagrams do not each reinvent geometry. A small set of shared kernels under `src/graph`, `src/text`, and `src/style` does the reusable work; diagram layout engines compose them. This is what keeps the per-diagram layout code short.

0.4.1 Layout (`src/graph`)

`layered.ts`
A Sugiyama-lite layered placement: longest-path ranking plus size-aware coordinate assignment. Used by the node-link and UML kinds (class, state, ER, flowchart) so they share rank assignment.

`tree.ts`
A tidy hierarchical placement: every parent is centred over the block its descendants occupy, sibling subtrees never overlap, and node sizes may vary. Used by the whole tree family.

`connect.ts`
Edge-endpoint helpers: `borderPoint` clips a connector to a node's border; `connectSlots` clips both ends along the line joining two boxes — the *slot-aware pointer* used for cell-to-cell, node-to-node, and panel-to-panel arrows alike.

0.4.2 Text (`src/text`)

`measureText` gives deterministic width/height for a string at a font size (layout must not depend on a live font engine), and `wrap`/truncate helpers handle multi-line and overflow.

0.4.3 Cells and strips (`src/scene/strip.ts`)

`buildStrip` produces a contiguous (or gapped) run of equal cells — arrays, heap array-backings, B-tree key strips, slotted pages — and returns the bounding rectangle of each cell as a *slot* so pointers can clip to individual cells.

0.4.4 Cost tiers (`src/style/cost.ts`)

A reusable styling layer maps a numeric edge weight (latency, hop distance, plan cost) onto a discrete visual tier — colour plus dash pattern — and builds a legend:

```
classifyCost(scale, weight): CostTier   // bucket a weight into a
    tier
buildLegend(pen, theme, scale, origin)  // render the tier legend
```

It is pure styling, no layout, and is consumed by the topology family (§0.5).

0.5 Diagram Families

A `DiagramKind` is one registered module. They fall into a Mermaid-compatible group and four Triton-native families. The detector recognises the Mermaid headers and the Triton-native ones alike (`src/frontend/detect.ts`).

0.5.1 Mermaid-compatible

These accept Mermaid syntax and render it unchanged, so existing diagrams migrate with zero edits:

```
flowchart LR
  start[Start] --> validate{Valid?}
  validate -->|yes| process[Process]
  validate -->|no|  reject[Reject]
```

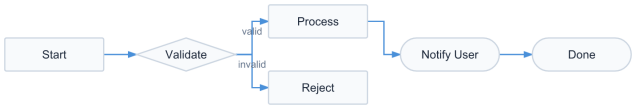

Figure 2: A Mermaid flowchart, rendered unchanged through the `flowchart` module.

The set spans node-link (flowchart), UML/software (sequence, class, state, ER), project (gantt, timeline), and chart kinds. The charts (pie, xychart, quadrant, radar) are *four separate modules*, not a single "grammar of graphics" — each owns the IR that fits it.

0.5.2 Tree family

A single decorated `tree` engine underlies the family. Its IR is a *flat, id-referenced node list* — not recursively nested nodes — where each node carries a small decoration vocabulary (shape, colour class, corner badge, sub-label, edge label). The tidy layout from §0.4 places it.

On top of that engine sit *value-driven* front-ends. Their source is a list of operations, and the builder computes a valid structure — you cannot author an invalid one:

```
avl insert 50 30 70 20 40 60 80 10
```

renders a balanced AVL tree with a balance-factor badge on every node, because the builder runs real insertion with rotations. The siblings work the same way: `rbtree` (red/black colouring), `btree order N` (multi-key strip nodes), `radix` (prefix-compressed tries with substring edge labels), `segtree over [...] reduce sum` (range-annotated nodes), `heap max` (sift-up). `plan` reuses the same engine for query-plan trees, inferring operator colour from the node label.

0.5.3 Struct / memory family

These exercise the cell-strip and slot-aware pointer kernels:

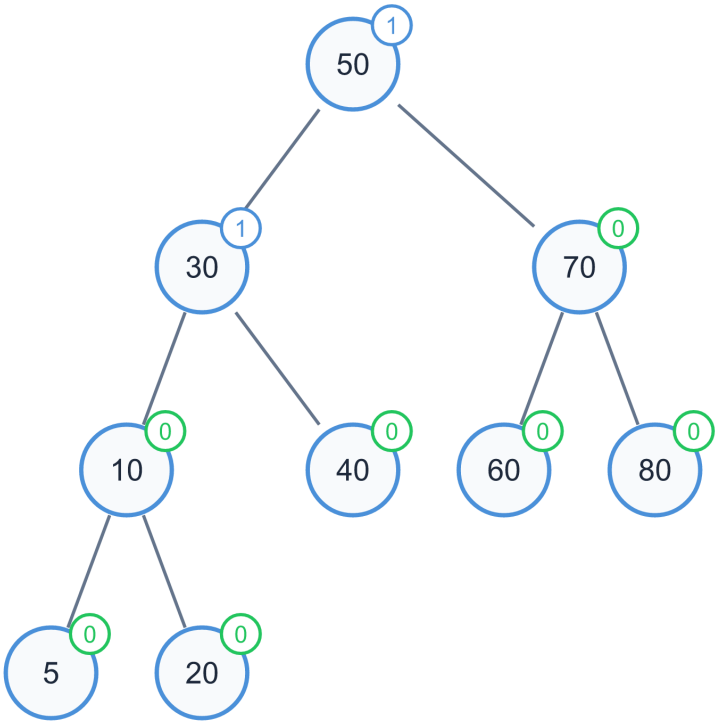

Figure 3: An AVL tree built from the `insert` list above. The balance-factor badge on each node is computed by running real insertion with rotations — the structure cannot be authored unbalanced.

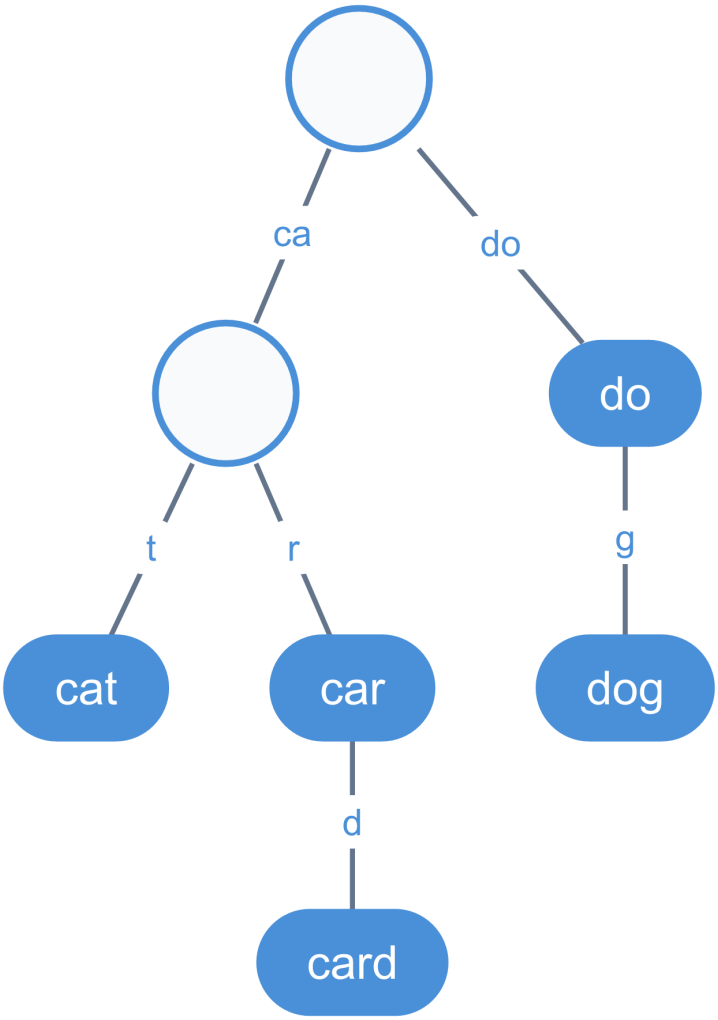

Figure 4: A radix (Patricia) trie over a set of words: shared prefixes are compressed onto single labelled edges.

```
array
  title int[] nums
  cells 5 8 13 21 34
  index
  ptr i -> 2
```

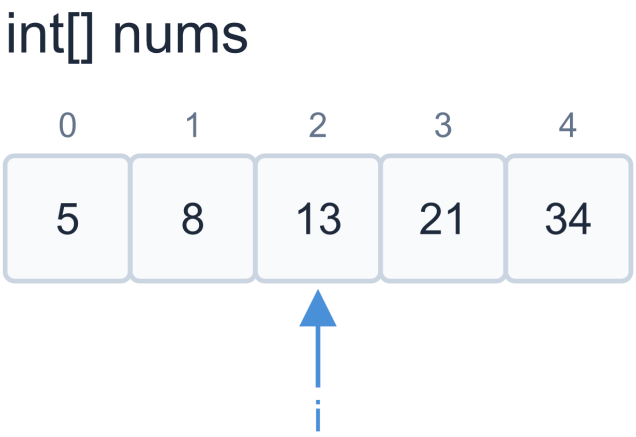

Figure 5: An array with an index row and a named pointer into a cell.

`linkedlist` draws a `head` chain of `[value|next]` nodes terminating in a null slash; `memory` lays out named regions (e.g. a stack and a heap) and routes a *cross-region pointer* from a variable slot to a heap object via `connectSlots`; `page` draws a slotted storage page whose line pointers indirect to tuples.

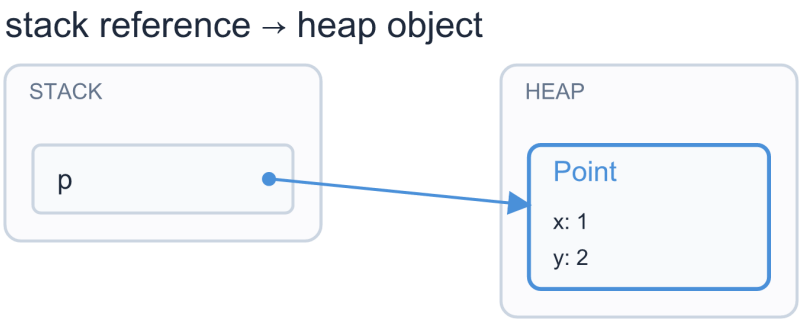

Figure 6: A memory model: a variable slot in one region holds a pointer that crosses into a heap object in another, routed by the slot-aware connector.

0.5.4 Topology

A cost-weighted node graph. Edges are coloured and dashed by the tier their weight falls into, a legend is generated from the declared scale, and nodes may be nested inside group containers:

```
topology
  title NUMA interconnect
  costs ns
    tier local 90 #27ae60
    tier hop1  140 #2f80ed
    tier hop2  200 #e2574c 5 4
  node N0 : Node 0
  node N1 : Node 1
  N0 -- N1 : 140
```

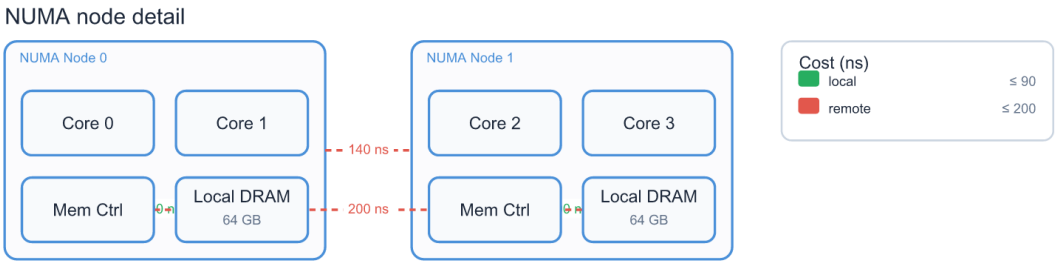

Figure 7: A NUMA topology with nested node groups: links are coloured and dashed by the cost tier their weight falls into, and the legend is generated from the declared scale.

0.6 Composition and Cross-Diagram Links

Triton's distinguishing capability is putting whole diagrams together on one canvas and drawing connections *between* them. Two features cooperate: the `poster` grid and cross-diagram links.

0.6.1 Posters

A `poster` arranges child diagrams in a grid. A cell may hold *any* registered diagram kind — the cell kind is dispatched through the same registry the top-level pipeline uses (`src/diagrams/poster/`), so a poster can embed a plan tree next to a B-tree next to a slotted page. Cells flow left-to-right and wrap at the column count; a cell may span columns or rows:

```
poster "Cell Spanning"
    columns 3

    cell hero "spans all 3 columns" [3] :: plan
        plan
            Hash Join {rows: 980}
                Seq Scan orders {rows: 10000}
    end

    cell idx "spans 2" [2] :: btree
        btree order 3 insert 10 20 30 40 50 60
    end
```

```
    cell k "fits the 3rd" :: stat
        13 | kinds
    end
```

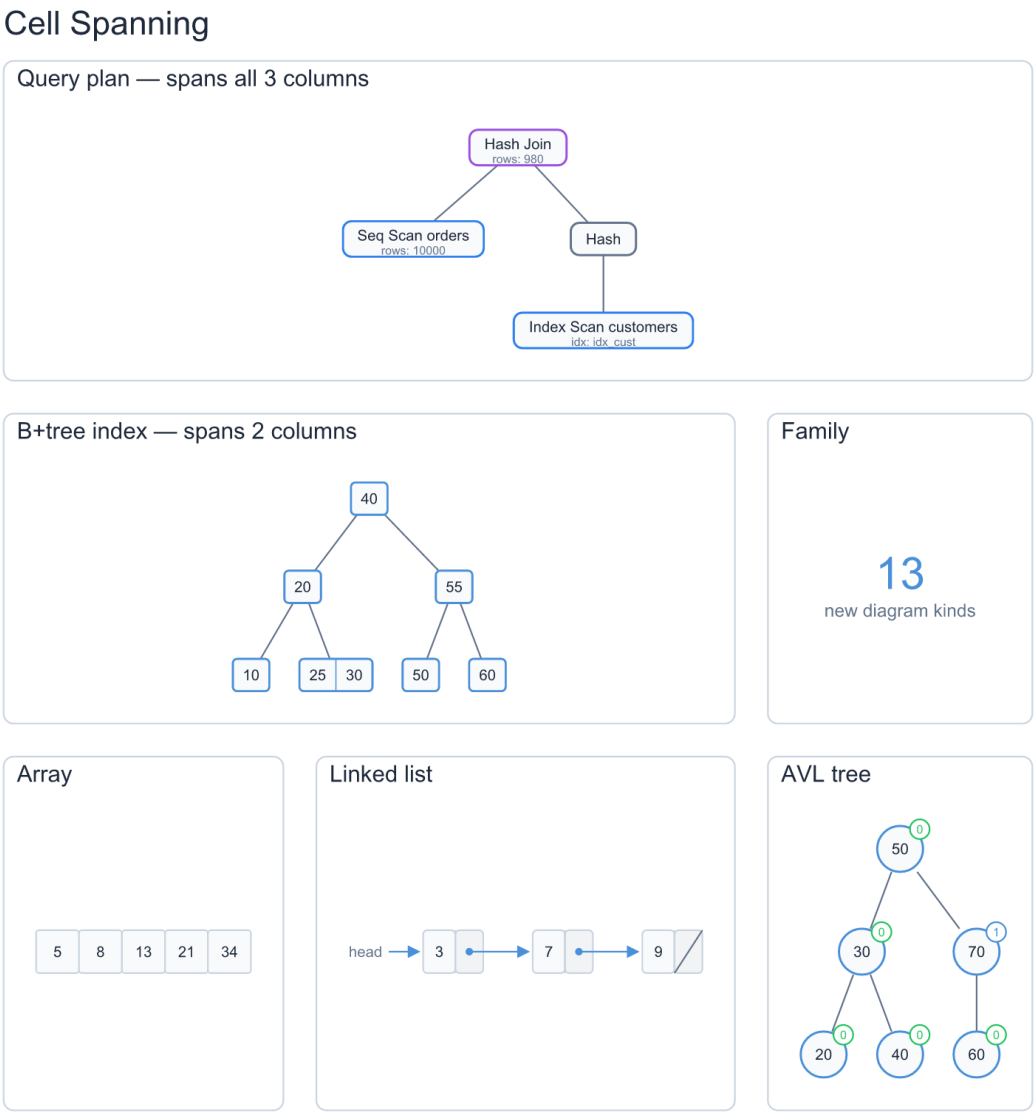

Figure 8: A poster demonstrating cell spanning: a plan tree spanning all three columns, a B-tree spanning two, and single-width cells flowing into the remaining slots.

`[n]` spans `n` columns; `[c x r]` spans `c` columns by `r` rows. Placement is occupancy-aware: a row-spanning cell reserves the rows it covers, so later cells flow past it without overlap.

0.6.2 Cross-diagram links

Because each diagram's `layout` returns a node-anchor registry (§0.2.2), the poster layer can address a node *inside* a child diagram as `cellId.nodeId` and route an arrow to a node in a different cell:

```
    link q.n3  --> idx.n0 "scan uses index"
```

```
    link idx.n0 --> pg.slot0 "leaf points to page"
```

The poster merges child anchor registries with their cell id as a path prefix, then routes the link with the same slot-aware connector used everywhere else. This is what lets, for example, an "SQL engine anatomy" poster show a query plan, a B-tree index, and a storage page as one connected story.

0.7 Themes and Determinism

0.7.1 Themes

Triton separates meaning from appearance. The grammar fixes what a diagram *is*; the theme fixes how it *looks*. A diagram never hard-codes a colour, and a theme never changes a diagram's structure.

A theme is one resolved token set (`src/contracts/theme.ts`) — `ResolvedTheme` — with token groups for palette, typography, spacing, edges, and panel chrome:

```
interface ResolvedTheme {
  palette:    ThemePalette;     // primary, surface, border, text,
     success, ...
  typography: ThemeTypography;  // font families, sizes, line height
  spacing:    ThemeSpacing;
  edges:      ThemeEdges;
  panel:      ThemePanel;       // titled-container chrome
}
```

A resolver merges a user override over a base preset; the codebase ships a dozen presets (`src/theme/preset.ts`). One resolved theme applies uniformly to *every* diagram kind — switching themes restyles a flowchart, a tree, and a poster identically, because they all read the same tokens through the `Pen`.

0.7.2 Determinism

The same source and theme always produce the same SVG, byte for byte. This is a property the whole pipeline is built to preserve, not an afterthought:

- Layout is a pure function of the IR and theme — no clocks, no random numbers, no hash-order iteration over unordered maps.
- Text measurement (`measureText`) is deterministic, so geometry does not depend on a live font engine.
- Numeric output is formatted at fixed precision, and elements are emitted in a stable document order.
- The value-driven builders are deterministic by construction: the same `insert` list yields the same tree.

Byte-stable SVG is what makes the output diff-able, cacheable, and safe to commit beside its source — which is exactly how the `examples/` directory is organised, each `.svg` regenerated from its `.mmd`.

0.8 Status and Scope

0.8.1 What is built

- The full pipeline: detection, registry dispatch, the `DiagramModule` contract, the `Scene` contract, and the SVG renderer (PNG via `resvg`).
- The Mermaid-compatible kinds and the four Triton-native families (tree, struct/memory, topology, poster composition) — roughly three dozen registered `DiagramKind`s in total.
- Poster composition with cell spanning and cross-diagram links.
- A dozen theme presets over one unified `ResolvedTheme`.
- A test suite of 318 passing tests covering grammars, layout invariants (e.g. AVL balance, B-tree key bounds, heap order), rendering, and a smoke test that renders every `examples/**/*.mmd` to valid SVG.

0.8.2 What is explicitly out of scope

These are *not* implemented and are not described elsewhere in this document as if they were:

- **No natural-language input.** Triton compiles source; it does not construct diagrams from prompts.
- **No data ingestion.** There are no source adapters that crawl external systems into an IR.
- **No PPTX / PDF / HTML / Skia backends.** The single output is SVG (rasterised to PNG when required).
- **No general animation pipeline.** The only animated effects are two SVG path overlays — marching ants (`march`) and particles (`particle`).
- **No agent/MCP ingestion API** and **no multi-package monorepo** — the project is one package.

0.8.3 Repository layout

Path	Contents
`src/contracts/`	Scene, DiagramModule, anchors/ports, theme contracts
`src/frontend/`	detect + registry dispatch
`src/diagrams/`	one folder per kind (grammar, IR, layout); the data-structure families are grouped und
`src/graph/`	layered, tree, connect layout kernels
`src/scene/`	Pen builder, cell-strip builder
`src/style/`	cost-tier styling
`src/theme/`	presets + resolver
`src/render/`	the SVG renderer
`examples/`	`.mmd` sources beside rendered `.svg`

0.8.4 Possible future work

Honest candidates, none promised: a vector PDF backend, richer auto-layout for dense graphs, and a first-class JSON/YAML authoring schema to make the structured input path as ergonomic as the text one.

0.9 LaTeX Integration

Triton diagrams can be embedded in LaTeX documents as *vector* PDF figures. The integration lives in its own satellite package, `latex/` (`@triton/latex`), and is purely additive: it changes no core behaviour and adds **zero** dependencies to the `triton` package.

0.9.1 The format gap

LaTeX's `\includegraphics` ingests PDF, PNG, or JPG — never SVG — on every engine (pdfLaTeX, XeLaTeX, LuaLaTeX). Triton emits SVG (§0.3: `renderSync` → `renderSVG`). The integration therefore converts each diagram to a PDF *ahead of time*, and the document includes the committed PDF. This "precompile and commit" model is the only one that also works on Overleaf, where no Node or Triton toolchain is available at compile time.

0.9.2 SVG → vector PDF, pure-JS

The converter is pure JavaScript with **no system binaries** (no Inkscape, no `rsvg-convert`, no headless browser). It feeds Triton's SVG through `pdfkit` + `svg-to-pdfkit`, producing a single-page PDF whose page box equals the diagram's `viewBox`. The output is genuinely vector:

- shapes become PDF path operators (`re`, `l`, `c`);
- the `Scene`'s five element types and the arrowhead `<marker>` definitions all replay faithfully;

- text becomes real vector glyphs in the embedded base-14 fonts (Helvetica / Times / Courier) — so there is *no font drift* between author, CI, and Overleaf, and no external font file to ship.

The SMIL animation overlays (`march`, `particle`) are dropped: a PDF is static, and the first frame is the correct print representation.

0.9.3 The CLI

The package ships a `triton-latex` command (esbuild-bundled from `latex/src/cli.ts`, which imports the compiler by relative path from `../src` — the same satellite pattern as the VS Code extension):

```
triton-latex render <input.triton|.mmd> -o <out.pdf|.svg> [--theme N] [--scale S]
triton-latex render-dir <srcDir>        -o <outDir>        [--theme N] [--scale S]
```

`render` converts one source to vector PDF (or passes SVG through); `render-dir` batch-renders every `*.triton`/`*.mmd` in a directory to `<name>.pdf`. Both reuse the core `renderSync()` path and surface its `Result` as a clean exit code.

0.9.4 The `triton.sty` package

A thin LaTeX package wraps `\includegraphics`, resolving a diagram by name against a configurable directory. It depends only on `graphicx`, so it is engine-agnostic.

```
\usepackage{triton}
\tritondir{figures}                % where the rendered <name>.pdf live
\triton{flowchart}                 % includegraphics figures/flowchart.pdf
\tritonfig[width=0.6\linewidth]{avl}{An AVL tree rendered by Triton.}
```

0.9.5 Workflow and portability

The authoring loop mirrors the design doc's own `pnpm figures` / `\ourfig` convention: render sources to PDF, commit them, include by name, and regenerate when a diagram changes. Because the assets are committed vector PDFs, the same project compiles unchanged on macOS, Linux, Windows, and Overleaf — the engine axis is irrelevant, since the asset is already a PDF by the time `\includegraphics` runs.

Path	Contents
`latex/src/cli.ts`	the `triton-latex` CLI (render / render-dir)
`latex/src/pdf.ts`	SVG → vector PDF (pdfkit + svg-to-pdfkit)
`latex/triton.sty`	the `\triton` / `\tritonfig` macros
`latex/esbuild.mjs`	bundles the CLI + compiler graph from `src/`
`latex/examples/`	`diagrams/` sources, committed `figures/` PDFs, `demo.tex`